



GUION MAESTRO Y DIRECTIVAS DE PRESENTACIÓN EN VIDEO

PROYECTO PITA — Programa Integrado de Transacciones Académicas (UPC)

Universidad Popular del Cesar — Facultad de Ingenierías y Tecnológicas — Ingeniería de Sistemas

Asignatura: Estructura de Datos | **Docente:** Ing. Adith Pérez Orozco | **Taller 1 — Listas Formato de Video:** ~10 minutos exactos. Los 3 integrantes con **cámara encendida TODO el tiempo**.

AVISO ANTES DE GRABAR: Preparen DOS escritorios virtuales (o dos monitores):

- **Escritorio izquierdo:** `pita_gui.exe` (C++ con Dear ImGui) — ventana grande a pantalla completa.
- **Escritorio derecho:** `python gui_main.py` (Python con CustomTkinter) — ventana grande a pantalla completa.
- **VS Code abierto** con `cpp/ dominio/ lista_enlazada.h` en tab 1 y `cpp/ gui/ gui_vista_nomina.cpp` en tab 2.
- **PowerShell** en la carpeta `C:\Users\Chick\Desktop\Taller` con los comandos ya escritos pero sin ejecutar.



SECCIÓN 1 — REGLAS DE ORO (LEA ANTES DE GRABAR)

#	Regla Obligatoria	Por qué es crítico
1	Cámara web encendida los 3 TODO el tiempo	El taller lo exige explícitamente. Un integrante sin cámara = riesgo de anular la presentación.
2	Saludo con identidad institucional UPC	El ítem "ii" del taller dice "debe respetarse la identidad institucional de la UPC".
3	Demostrar AMBOS programas (C++ y Python)	El ítem "22" del taller exige literalmente dos implementaciones: C++ y Python.
4	Entre 9:30 y 10:30 minutos	El ítem "i" del taller dice "alrededor de 10 minutos". Nunca menos de 9 ni más de 11.
5	Cada estudiante habla de su bloque asignado	El profesor puede pedir a cualquiera que explique más. Cada quien defiende solo lo suyo.



SECCIÓN 2 — DISTRIBUCIÓN DE ROLES (Calibrada para compensar diferencia de nivel)

Rol	Estudiante	Segmento	Nivel de Dificultad Técnica	Qué Muestra
Apertura e Institucional	Estudiante 2 (falencias A)	Min 0:00 – 3:00	★ Bajo	Dashboard, Facultades, Personas, Persistencia. Todo visual, sin código.
Académico y Contratos	Estudiante 3 (falencias B)	Min 3:00 – 6:00	★★ Medio	Cursos, Ofertas, Matrículas, EBRA, Contratos. Flujo de clics en GUI.
TAD Lista + Nómina Legal + Cierre	Estudiante 1 (el fuerte)	Min 6:00 – 10:00	★★★ Alto	Código C++ de listas, motor Dec. 1279 / Ac. 027, comparación C++ vs Python, tests.

Estrategia clave: El Estudiante 1 hace el cierre técnico profundo. Los compañeros solo navegan la GUI y recitan su libreto. Si Adith pregunta algo técnico a los compañeros, pueden decir: *"Con más detalle lo explica mi compañero"*

[Nombre Estudiante 1]".

SECCIÓN 3 — GUION MINUTO A MINUTO (CON NAVEGACIÓN EXACTA)

BLOQUE 1 — APERTURA, MODELO UPC, FACULTADES, PERSONAS Y PERSISTENCIA

Orador: Estudiante 2 | **Tiempo:** 0:00 a 3:00 (180 seg) **Pantalla al comenzar:** `pita_gui.exe` en C++ — Vista **Dashboard**

Segmento 0:00 — 0:45 | Saludo institucional y propósito del software

Qué mostrar en pantalla: GUI de C++ en el Dashboard, sin tocar el mouse. Dejar que los KPIs se vean de fondo.

Libreto (leer / memorizar):

"Muy buenos días, profesor Adith Pérez Orozco y compañeros. Somos el Grupo [#] de la asignatura Estructura de Datos de la Universidad Popular del Cesar, Facultad de Ingenierías y Tecnológicas. Nuestro equipo está conformado por [Nombre 1], [Nombre 3] y mi persona [Nombre 2].

Hoy presentamos el **Programa Integrado de Transacciones Académicas — PITA**, desarrollado para reemplazar el Vortal de la UPC. Lo implementamos en **dos lenguajes completamente distintos**: una versión en **C++** con interfaz gráfica de alto rendimiento usando la biblioteca *Dear ImGui* — que es la que ven ahora en pantalla — y una versión equivalente en **Python** con la interfaz *CustomTkinter*.

Ambas versiones comparten exactamente la misma lógica de negocio, la misma normatividad legal y la misma arquitectura de datos."

Segmento 0:45 — 1:30 | Facultades y Programas Académicos

Acción de mouse: Clic en la barra lateral izquierda → botón **"Facultades"**. **Qué mostrar:** La tabla de facultades (Ej.: *Facultad de Ingenierías y Tecnológicas, Facultad de Derecho*) y los badges de estado. Luego hacer clic en el botón **"Ver Programas"** de una facultad para mostrar sus programas (Ej.: *Ingeniería de Sistemas*).

Libreto:

"En la vista de Facultades observamos que el sistema permite registrar, consultar, modificar y eliminar cada facultad de la universidad. Cada facultad es la cabecera de un conjunto de programas académicos, como podemos ver aquí para la Facultad de Ingenierías.

El sistema valida la integridad referencial: si intento eliminar una facultad que tiene programas activos vinculados, el sistema lo bloquea y lanza una advertencia. Esta protección es fundamental para garantizar que no haya datos huérfanos en la base de datos.

En la barra lateral pueden observar que ambas versiones — Python y C++ — tienen exactamente las mismas vistas: *Facultades, Personas, Académica, Contratos, Nómina y Parámetros*."

Segmento 1:30 — 2:30 | Personas: Docentes, Estudiantes y Administrativos

Acción de mouse: Clic en la barra lateral → **"Personas"**. **Qué mostrar:** La tabla de personas con los badges de color:  Docente,  Estudiante,  Administrativo. Hacer clic en el botón **"Ver Detalle"** de un docente para mostrar su ficha completa (nombre, documento, correo institucional, tipo). Si hay tiempo, abrir el modal **"Nueva Persona"** para mostrar los campos de entrada (no es necesario guardar nada).

Libreto:

"En la vista de Personas gestionamos toda la comunidad universitaria desde un único módulo. Cada persona tiene su tipo de documento, número de documento único, correo institucional y su rol dentro de la universidad — docente, estudiante o administrativo.

Como pueden ver, los badges de color identifican visualmente el rol: en azul los docentes, en verde los estudiantes, en ámbar los administrativos. El sistema garantiza que no existan dos personas con el mismo número de documento.

Cuando se registra un docente, adicionalmente se capturan su régimen salarial, dedicación horaria, categoría académica y puntos salariales reconocidos para el cálculo de nómina."

Segmento 2:30 — 3:00 | Persistencia de datos y transición

Acción de mouse: Abrir el explorador de Windows y navegar a `C:\Users\Chick\Desktop\Taller\datos\` para mostrar los archivos JSON que el sistema genera automáticamente (facultades.json, personas.json, contratos.json, etc.). También puede mostrar brevemente `persistencia/repositorios.py` en Python y mencionar el equivalente en C++ que son los mismos archivos.

Libreto:

"El taller exige que el programa cargue y guarde los datos en archivos. Aquí podemos ver la carpeta `datos/` con todos los archivos JSON que el sistema escribe automáticamente al cerrar y lee al arrancar. Tanto la versión Python como la C++ comparten exactamente esta misma carpeta y estos mismos archivos — son 100% compatibles.

Al iniciar, el usuario puede decidir cargar los datos existentes o empezar con una base de datos vacía, cumpliendo exactamente el punto 4.8 del taller.

Ahora mi compañero [Nombre Estudiante 3] les mostrará el ciclo académico del sistema."

BLOQUE 2 — CICLO ACADÉMICO COMPLETO, ALERTA EBRA Y CONTRATOS

Orador: Estudiante 3 | **Tiempo:** 3:00 a 6:00 (180 seg) **Pantalla al comenzar:** `pita_gui.exe` en C++ — Vista **Académica**

Segmento 3:00 — 3:45 | Catálogo de Cursos y Malla Curricular

Acción de mouse: Clic en "Académica" en la barra lateral. La vista tiene pestañas — ir a la pestaña "Catálogo de Asignaturas". Mostrar la tabla de cursos (Cálculo I, Programación II, Estructura de Datos, etc.) con sus créditos, horas teóricas y horas prácticas. Hacer clic en "Editar" de un curso para mostrar el modal con los campos editables (créditos, nota mínima aprobatoria, cupo).

Libreto:

"Buenas, continuando con el sistema. En la vista Académica gestionamos todo el ciclo de vida educativo de la universidad.

En el Catálogo de Asignaturas podemos ver cada curso con su código, nombre, número de créditos, horas teóricas, horas prácticas y la nota mínima aprobatoria. Desde aquí podemos editar cualquier parámetro del curso o eliminarlo con validación: el sistema no permite eliminar un curso que esté activo en la malla curricular o que tenga matrículas vigentes.

La pestaña de Malla Curricular muestra qué cursos pertenecen a cada semestre de cada programa académico, junto con sus prerrequisitos."

Segmento 3:45 — 4:30 | Ofertas de Cursos y Matrículas

Acción de mouse: Clic en la pestaña "Ofertas y Grupos" dentro de la vista Académica. Mostrar la tabla de ofertas (grupo A de Estructura de Datos, grupo B de Programación) con el docente asignado y el cupo disponible. Luego clic

en la pestaña "**Matrículas**" y mostrar la lista de estudiantes inscritos en cada oferta.

Libreto:

"En la pestaña de Ofertas, cada semestre se abre un grupo por curso con un cupo máximo definido, un aula, una modalidad y un docente responsable asignado. El sistema controla en tiempo real el cupo disponible.

Cuando un estudiante se matricula en un curso, el cupo disminuye automáticamente. Si el estudiante cancela la matrícula, el cupo se libera de inmediato. El sistema también valida que el estudiante no supere el máximo de créditos permitidos por período académico.

[Hacer clic en el botón 'Nueva Matrícula' para mostrar el modal de inscripción, aunque no se necesite guardar.]"

Segmento 4:30 — 5:20 | Calificaciones y Alerta EBRA (Punto crítico del taller)

IMPORTANTE: Este es uno de los puntos que Adith califica directamente — el punto 12 del taller lo exige explícitamente. **Acción de mouse:** Clic en la pestaña "**Calificaciones y EBRA**" dentro de la vista Académica. Señalar en la tabla la columna de **Promedio Acumulado** de un estudiante que tenga valor < 3.0 . Mostrar el **badge rojo de ALERTA EBRA** que aparece al lado de ese estudiante. Si hay tiempo, hacer clic en "**Editar Nota**" para mostrar el slider de 0.0 a 5.0.

Libreto:

"En el módulo de Calificaciones, el sistema registra las notas de cada evaluación y recalcula automáticamente el promedio ponderado del estudiante según los créditos de cada materia.

*Cumpliendo el requerimiento número doce del taller, implementamos el sistema de **Alertas Tempranas EBRA** — Estudiante de Bajo Rendimiento Académico. Como podemos ver aquí [señalar con el cursor], cuando un estudiante tiene un promedio acumulado inferior a 3.0, el sistema activa automáticamente este badge rojo de ALERTA EBRA. Esta alerta le permite a la dirección del programa intervenir oportunamente para apoyar al estudiante antes de que pierda el período.*

El recálculo es reactivo: en cuanto se registra o modifica una nota, el promedio y el estado EBRA se actualizan instantáneamente sin necesidad de ninguna acción adicional."

Segmento 5:20 — 6:00 | Contratos Docentes y transición

Acción de mouse: Clic en "**Contratos**" en la barra lateral. Mostrar la tabla de contratos con las tres modalidades: badge verde **PLANTA**, badge morado **OCASIONAL**, badge naranja **CÁTEDRA**. Hacer clic en "**Detalle**" de un contrato para mostrar la ficha técnica (tipo de contrato, dedicación, sueldo base, fechas de vigencia, régimen ARL).

Libreto:

*"Finalmente en mi bloque, el módulo de Contratación Docente permite gestionar los tres tipos de vinculación que exige la normatividad colombiana: profesores de **Planta** — vinculación indefinida, escala salarial por puntos —, **Ocasionales** — contrato por períodos—, y **Catedráticos** — pagados por hora.*

El sistema valida automáticamente las restricciones legales: un docente catedrático no puede superar las 18 horas semanales según el Acuerdo 027 de la UPC. Si se intenta asignar más, el sistema bloquea la operación con un mensaje de advertencia.

Le cedo la palabra a mi compañero [Nombre Estudiante 1] para que explique la estructura de datos en C++ y el motor de nómina."

BLOQUE 3 — TAD LISTA ENLAZADA EN C++, MOTOR DE NÓMINA LEGAL Y COMPARACIÓN C++ vs PYTHON

Orador: Estudiante 1 (Líder Técnico) | **Tiempo:** 6:00 a 10:00 (240 seg) **Estrategia de navegación:** Este bloque tiene 4 sub-segmentos con cambio de ventana en cada uno.

Segmento 6:00 — 7:15 | El TAD Lista Enlazada en C++ (La Estructura de Datos Propia)

Acción: Cambiar al VS Code, abrir `cpp/ dominio/ lista_enlazada.h`. **Mostrar línea 11-19:** La declaración del `struct Nodo<T>` con los punteros `anterior` y `siguiente`. **Mostrar líneas 22-26:** La clase `ListaEnlazada<T>` con los miembros privados `cabeza`, `cola`, `longitud`. **Mostrar líneas 166-176:** El método `push_back` con la manipulación manual de punteros. **Mostrar línea 154-156:** El destructor `~ListaEnlazada()` que libera la memoria.

Libreto:

"Para fundamentar todo el sistema en C++, diseñamos desde cero el **Tipo Abstracto de Dato Lista Enlazada Doble Genérica** — sin usar `std::List` ni ningún contenedor de la librería estándar — como lo requiere la materia de Estructura de Datos.

[Señalar líneas 11-19 en pantalla] Aquí vemos el `struct Nodo<T>`: cada nodo tiene el dato de tipo genérico `T`, un puntero al nodo `anterior` y un puntero al nodo `siguiente`. Esto es la lista doblemente enlazada pura.

[Señalar líneas 22-26] La clase `ListaEnlazada` mantiene un puntero `cabeza` al primer nodo y un puntero `cola` al último, más un contador `longitud`. Con estos tres elementos controlamos toda la lista sin arrays.

[Señalar líneas 166-176] El método `push_back` crea un nuevo nodo con `new Nodo<T>`, conecta los punteros `siguiente` de la cola anterior hacia el nuevo nodo, y el puntero `anterior` del nuevo nodo hacia la cola anterior. Así crece la lista. Y [señalar líneas 154-156] el destructor libera cada nodo con `delete` para no tener fugas de memoria.

Esta misma estructura se usa para almacenar Facultades, Personas, Cursos, Contratos, Liquidaciones — absolutamente todo. Cada colección del sistema es un `ListaEnlazada<TipoDeDato>`."

Segmento 7:15 — 8:00 | Nómina Legal: Decreto 1279 vs Acuerdo 027

Acción: Cambiar a `pita_gui.exe` en C++ — hacer clic en "Nómina" en la barra lateral. Mostrar la tabla principal de **Resumen de Liquidaciones** con los badges de tipo (PLANTA, OCASIONAL, CÁTEDRA). Señalar las columnas: **Sueldo Básico**, **Devengado**, **Descuentos Ley**, **Neto a Pagar**, **Prestaciones**. Luego abrir el botón "Liquidar Empleado" para mostrar el modal con selector de período, tipo de personal y combo de empleados.

Libreto:

"Ahora el componente más riguroso del proyecto: el **Subsistema de Nómina**, que implementa con exactitud matemática el **Decreto 1279 de 2002** y el **Acuerdo 027 del 31 de octubre de 2024** de la UPC.

El principio central es el **aislamiento de regímenes salariales**:

- Para los **Docentes de Planta**: el salario se calcula multiplicando los puntos salariales reconocidos por el valor del punto vigente — actualmente \$23.924 pesos. Los puntos acumulan por título académico, categoría, experiencia y productividad investigativa.
- Para los **Docentes Ocasionales y Catedráticos**: la norma prohíbe asignarles puntos del Decreto 1279. Su salario se fija según la tabla del Acuerdo 027, y si tienen doctorado, reciben una bonificación mensual no salarial equivalente al **90% de 1 SMLLV** — actualmente \$1.575.815 pesos. Esta bonificación no se mezcla con los puntos salariales de planta.

[Señalar la columna Descuentos Ley en la tabla] En las deducciones, el sistema aplica rigurosamente: **4% de salud** y **4% de pensión** a cargo del trabajador sobre el Ingreso Base de Cotización. Si el docente devenga 4 o más salarios mínimos, se aplica adicionalmente el **1% del Fondo de Solidaridad Pensional**.

El **Auxilio de Transporte** — \$249.095 pesos — solo se incluye automáticamente si la asignación básica no supera los 2 SMMLV. El sistema lo evalúa en tiempo real."

Segmento 8:00 — 8:45 | Parafiscales, Desglose Anual y Liquidación Individual

Acción: En la Vista Nómina de C++, clic en la pestaña **"Aportes Patronales & Parafiscales"**. Mostrar la tabla con Salud 8.5%, Pensión 12%, ARL 0.522%, SENA 2%, ICBF 3%, Caja 4%. Luego clic en la pestaña **"Desglose de Nómina Anual"** y mostrar la proyección a 12 meses. Finalmente hacer clic en el botón **"Liquidar Empleado"** para mostrar el modal equiparado: selector de período, radio buttons Docente/Administrativo, combo de empleados con badge [ **Contrato Activo**] o [ **Sin Contrato**].

Libreto:

*"[Señalar tabla de Parafiscales] El sistema también calcula la **carga patronal** que asume la universidad: Salud 8.5%, Pensión 12%, Riesgos Laborales ARL, SENA 2%, ICBF 3% y Caja de Compensación Familiar 4% — todo sobre el Ingreso Base de Cotización total del período.*

*[Cambiar a la pestaña Desglose Anual] La pestaña de **Desglose Anual** proyecta mes a mes los doce meses del año para cualquier contrato individual o para el consolidado institucional completo. Incluye el pasivo prestacional — cesantías, prima de servicios y vacaciones — que la universidad debe provisionar mensualmente.*

*[Mostrar el modal Liquidar Empleado] Con el botón **'Liquidar Empleado'** podemos liquidar individualmente a un docente o un administrativo en el período de nómina que seleccionemos. El sistema verifica en tiempo real si ese empleado tiene un contrato activo — si no lo tiene, bloquea la liquidación con una alerta. Si el período está cerrado, también bloquea la emisión de nuevas liquidaciones para preservar la auditoría."*

Segmento 8:45 — 9:30 | Comparación C++ vs Python en vivo

ESTE ES EL MOMENTO MÁS PODEROSO DEL VIDEO — muestra que SON DOS PROGRAMAS INDEPENDIENTES.

Acción: Cambiar al escritorio/ventana con `gui_main.py` (Python CustomTkinter). Navegar a la misma Vista de Nómina. Mostrar en Python la misma tabla de liquidaciones, el mismo modal de **Liquidar Empleado** y los mismos cálculos. Luego alternar entre ambas ventanas (Alt+Tab visible) para que el profesor vea que son dos aplicaciones distintas ejecutándose en simultáneo.

Libreto:

"Ahora les muestro la paridad completa. [Alternar entre `pita_gui.exe` y `gui_main.py`]

*Esta es la versión en **Python con CustomTkinter** — una aplicación completamente independiente en un lenguaje completamente diferente. Sin embargo, navega exactamente por las mismas vistas: Facultades, Personas, Académica, Contratos, Nómina y Parámetros.*

[Mostrar la vista de Nómina en Python con su tabla de liquidaciones] Los mismos valores de liquidación, los mismos cálculos del Decreto 1279, los mismos parafiscales. Esto es posible porque ambas implementaciones comparten la misma lógica de negocio — el motor matemático de nómina está replicado fielmente en C++ y en Python.

*La diferencia fundamental está en la **estructura de datos interna**: en Python usamos listas nativas del lenguaje, mientras que en C++ implementamos desde cero los nodos con punteros como vimos en el código.*

*Otro punto diferencial: en Python la interfaz es multiplataforma con CustomTkinter. En C++, usamos **Dear ImGui** — un renderizador inmediato de alta performance que dibuja la UI directamente sobre OpenGL, diseñado para aplicaciones de tiempo real."*

Segmento 9:30 — 10:00 | Tests automatizados y cierre formal

Acción: Cambiar a la PowerShell en la carpeta raíz del proyecto. Ejecutar: `.\cpp\build\Release\pita_backend.exe` y mostrar que terminan con el mensaje **TODAS LAS PRUEBAS DEL BACKEND C++ COMPLETADAS CON ÉXITO**. Luego ejecutar `pytest -v` y mostrar la pantalla de **47 passed in 1.66s**.

Libreto:

"[Ejecutar pita_backend.exe] Para cerrar, ejecutamos el banco de pruebas del backend C++. Como pueden ver, las 5 suites de pruebas — que incluyen integridad referencial, reglas del Decreto 1279, aislamiento de regímenes, liquidación completa y desglose anual — pasan todas al 100%.

[Ejecutar pytest] Y en Python, los 47 tests unitarios de integración y validación legal pasan sin ninguna falla.

*Con esto concluimos la presentación del **Programa Integrado de Transacciones Académicas PITA** de la Universidad Popular del Cesar. Implementamos los dos programas exigidos — C++ y Python —, con estructuras de datos propias, persistencia en archivos, gestión académica completa, alertas EBRA, contratación docente y un motor de nómina estrictamente apegado al Decreto 1279 de 2002 y al Acuerdo 027 de 2024.*

Muchas gracias por su atención profesor Adith y compañeros. Quedamos atentos a sus preguntas."

 SECCIÓN 4 — GUÍA DE NAVEGACIÓN EXACTA (PANTALLA POR PANTALLA)

Tiempo	Orador	Aplicación Activa	Vista/Archivo en Pantalla	Acción Exacta con el Mouse
0:00–0:45	Estudiante 2	C++ pita_gui.exe	Dashboard (vista inicial al abrir)	Sin tocar mouse. Hablar mientras el Dashboard se ve de fondo.
0:45–1:30	Estudiante 2	C++ pita_gui.exe	Vista Facultades	Clic en "Facultades" en la barra lateral → Clic en "Ver Programas" de una fila.
1:30–2:30	Estudiante 2	C++ pita_gui.exe	Vista Personas	Clic en "Personas" → Clic en "Ver Detalle" de un docente → Mostrar modal de ficha.
2:30–3:00	Estudiante 2	Explorador de Windows	Carpeta Taller/datos/	Abrir carpeta, mostrar los archivos .json de persistencia (facultades.json, personas.json...).
3:00–3:45	Estudiante 3	C++ pita_gui.exe	Vista Académica → pestaña "Catálogo de Asignaturas"	Clic en "Académica" → pestaña cursos → clic "Editar" en un curso para mostrar el modal.
3:45–4:30	Estudiante 3	C++ pita_gui.exe	Vista Académica → pestaña "Ofertas y Grupos"	Clic en la pestaña "Ofertas" → mostrar tabla con docente asignado → clic en pestaña "Matrículas".
4:30–5:20	Estudiante 3	C++ pita_gui.exe	Vista Académica → pestaña "Calificaciones y EBRA"	Clic en la pestaña "Calificaciones" → señalar badge rojo EBRA de un estudiante con promedio < 3.0.
5:20–6:00	Estudiante 3	C++ pita_gui.exe	Vista Contratos	Clic en "Contratos" → Clic en "Detalle" de un contrato para ver la ficha técnica.
6:00–7:15	Estudiante 1	VS Code	Archivo cpp/dominio/lista_enlazada.h	Abrir VS Code → mostrar líneas 11-19 (Nodo), líneas 22-26 (cabeza/cola), líneas 166-176 (push_back), líneas 154-156 (destructor).
7:15–8:00	Estudiante 1	C++ pita_gui.exe	Vista Nómina → pestaña "Resumen de Liquidaciones"	Clic en "Nómina" → mostrar tabla con badges PLANTA/OCASIONAL/CÁTEDRA →

Tiempo	Orador	Aplicación Activa	Vista/Archivo en Pantalla	Acción Exacta con el Mouse
				señalar columnas Devengado/Descuentos/Neto.
8:00– 8:45	Estudiante 1	C++ <code>pita_gui.exe</code>	Vista Nómina → pestaña "Aportes Patronales"	Clic en pestaña "Aportes Patronales" → mostrar tabla parafiscales → clic "Desglose Anual" → clic botón "Liquidar Empleado" para mostrar modal.
8:45– 9:30	Estudiante 1	Python <code>gui_main.py</code>	Vista Nómina en Python	Alt+Tab a Python → navegar a Nómina → mostrar misma tabla de liquidaciones → Alt+Tab de vuelta a C++ para comparar.
9:30– 10:00	Estudiante 1	PowerShell	Terminal raíz del proyecto	Ejecutar <code>.\cpp\build\Release\pita_backend.exe</code> → esperar resultado → ejecutar <code>pytest -v</code> → mostrar "47 passed".

SECCIÓN 5 — BANCO DE PREGUNTAS TRAMPA DE ADITH (Con respuestas exactas)

Para Estudiante 2 — Tema: Persistencia y Listas

Pregunta posible de Adith: "¿Cómo están guardando los datos y qué pasa si el programa se cierra de golpe?"

Respuesta:

"Profesor, el sistema serializa todas las listas enlazadas a archivos JSON en la carpeta `datos/`. Cada vez que el usuario realiza una operación de creación, edición o eliminación, el sistema guarda automáticamente. Al volver a iniciar, carga el archivo y reconstruye cada lista enlazada nodo por nodo. Si el programa se cierra de golpe antes de guardar la última operación, la información de esa operación podría no persistir, pero los datos anteriores están seguros porque ya fueron escritos al disco."

Para Estudiante 3 — Tema: EBRA y Matrículas

Pregunta posible de Adith: "¿Cómo calculan el promedio y por qué EBRA es menor a 3.0 y no a 2.5?"

Respuesta:

"Profesor, el promedio se calcula de forma ponderada: sumamos (nota de cada curso × créditos del curso) y dividimos entre el total de créditos cursados. El umbral de 3.0 lo impone el Reglamento Estudiantil de la UPC vigente — el sistema lee ese valor desde los parámetros normativos configurables, por lo que si la universidad lo cambia, el sistema se actualiza sin necesidad de tocar código."

Para Estudiante 1 — Tema: Lista Enlazada y Nómina

Pregunta posible de Adith: "¿Por qué usar lista enlazada y no un arreglo o vector?"

Respuesta:

"Profesor, la lista enlazada es superior para este dominio porque las colecciones de la universidad tienen tamaño dinámico — no sabemos de antemano cuántos profesores, estudiantes o liquidaciones habrá. La lista enlazada crece y decrece en $O(1)$ sin reasignar bloques contiguos de memoria como lo haría un arreglo. Además, la eliminación en el medio es $O(n)$ pero sin el costo de desplazar elementos. En el contexto académico y de nómina, la eficiencia de inserción y eliminación dinámica supera las ventajas de acceso aleatorio de los arreglos."

Pregunta posible de Adith: "¿Qué diferencia hay entre la nómina de un profesor Ocasional y uno de Planta en su sistema?"

Respuesta:

"Profesor, la diferencia es de régimen normativo. Un docente de Planta liquida su salario multiplicando los puntos salariales reconocidos por el valor del punto vigente — actualmente \$23.924. Un Ocasional NO acumula puntos del Decreto 1279; su salario lo fija la tabla del Acuerdo 027 según su nivel de dedicación. Además, si el Ocasional tiene doctorado, recibe una bonificación mensual no salarial del 90% del SMMLV, mientras que el docente de Planta con doctorado recibe esos puntos acumulados en su escala salarial permanente. Nuestro sistema aísla completamente estos dos regímenes para evitar cruzar cálculos entre ellos."

SECCIÓN 6 — CHECKLIST TÉCNICO PREVIO A LA GRABACIÓN

30 minutos antes de grabar:

- ☐ **Compilar en Release:** Ejecutar en PowerShell: `cmake --build cpp/build --config Release` y verificar que termina sin errores.
- ☐ **Verificar tests:** Ejecutar `.\cpp\build\Release\pita_backend.exe` (debe decir "TODAS LAS PRUEBAS COMPLETADAS CON ÉXITO") y `pytest -v` (debe decir "47 passed").
- ☐ **Abrir todas las ventanas** y organizarlas:
 - `pita_gui.exe` a pantalla completa.
 - `python gui_main.py` minimizado (listo para Alt+Tab en el segmento 8:45).
 - VS Code con `cpp/ dominio/ lista_enlazada.h` en tab 1 abierto (listo para segmento 6:00).
 - PowerShell con los comandos ya escritos pero sin ejecutar (listo para segmento 9:30).
- ☐ **Probar micrófonos** de los 3 (sin eco, sin saturación, audio claro).
- ☐ **Cámaras web** de los 3 encendidas, con buena iluminación en el rostro.
- ☐ **Cronómetro** en mano de uno de los integrantes para controlar el tiempo de cada bloque.
- ☐ **Practicar una vez** en seco sin grabar para sincronizar los cambios de pantalla con los cambios de orador.

SECCIÓN 7 — EXPLICACIÓN COMPLETA DEL CÓDIGO (GUIA TÉCNICA PARA EL VIDEO)

Sí, se explica código. El taller es de *Estructura de Datos*. El profesor Adith espera ver las estructuras reales. Esta sección le da a Estudiante 1 el libretto exacto para cada fragmento de código que debe mostrar en pantalla, ordenado de mayor a menor importancia.

7.1 — ARCHIVO PRINCIPAL: `cpp/ dominio/ lista_enlazada.h`

¿Por qué es el archivo más importante? Aquí está el TAD Lista Enlazada implementado desde cero. Es el corazón del sistema en C++.

CÓDIGO 1 — La estructura del Nodo (Líneas 11–19)

```
// Abrir en VS Code: cpp/ dominio/ lista_enlazada.h → líneas 11-19
template <typename T>
struct Nodo {
    T dato;
    Nodo* anterior;
    Nodo* siguiente;

    explicit Nodo(const T& valor) : dato(valor), anterior(nullptr), siguiente(nullptr) {}
};
```

Libreto para decir mientras se muestra este código:

"Este es el bloque fundamental de nuestra lista: el `struct Nodo`. Cada nodo es un cajón que guarda un dato de tipo genérico `T` — puede ser una Facultad, un Profesor, una Liquidación, lo que sea. Además, cada nodo tiene dos punteros: `anterior` apunta al nodo que lo precede, y `siguiente` apunta al que viene después. Cuando `anterior` es `nullptr`, ese es el primer nodo — la `cabeza`. Cuando `siguiente` es `nullptr`, ese es el último — la `cola`. Así se construye la cadena de memoria dinámica."

CÓDIGO 2 — La clase ListaEnlazada y sus atributos privados (Líneas 22–26)

```
// Líneas 22-26
template <typename T>
class ListaEnlazada {
private:
    Nodo<T>* cabeza;    // Apunta al primer nodo
    Nodo<T>* cola;      // Apunta al último nodo
    size_t longitud;    // Contador de elementos
```

Libreto:

"La clase `ListaEnlazada` es un template genérico — funciona para cualquier tipo de dato. Internamente solo guarda tres cosas: el puntero `cabeza` al primer nodo, el puntero `cola` al último, y `longitud` como contador. Con solo esos tres datos controlamos una lista de tamaño arbitrario sin reservar memoria de más."

CÓDIGO 3 — Inserción: `push_back` (Líneas 166–176)

```
// Líneas 166-176
void push_back(const T& valor) {
    Nodo<T>* nuevo = new Nodo<T>(valor); // Crear nodo en memoria heap
    if (empty()) {
        cabeza = cola = nuevo;           // Lista vacía: nuevo es cabeza y cola
    } else {
        cola->siguiente = nuevo;          // Vieja cola apunta al nuevo
        nuevo->anterior = cola;           // Nuevo apunta atrás hacia la vieja cola
        cola = nuevo;                    // Actualizar cola al nuevo
    }
    longitud++;
}
```

Libreto:

"El método `push_back` inserta un elemento al final. Crea un nuevo nodo con `new` — asignando memoria dinámica en el heap. Si la lista está vacía, ese nodo es simultáneamente `cabeza` y `cola`. Si ya tiene elementos, conectamos la antigua `cola` hacia el nuevo nodo con `cola->siguiente = nuevo`, y conectamos el nuevo nodo de vuelta con `nuevo->anterior = cola`. Finalmente actualizamos `cola` al nuevo nodo. Costo: $O(1)$ — constante sin importar cuántos elementos tenga la lista."

CÓDIGO 4 — Eliminación: `remove_if` con predicado (Líneas 311–340)

```
// Líneas 311-340 — El método más sofisticado del TAD
template <typename Predicate>
bool remove_if(Predicate pred) {
    bool eliminado = false;
    Nodo<T>* actual = cabeza;
    while (actual != nullptr) {
        if (pred(actual->dato)) {
            Nodo<T>* a_borrar = actual;
            actual = actual->siguiente;
            // Reconectar los vecinos
            if (a_borrar->anterior)
                a_borrar->anterior->siguiente = a_borrar->siguiente;
            else
                cabeza = a_borrar->siguiente; // Era la cabeza
            if (a_borrar->siguiente)
                a_borrar->siguiente->anterior = a_borrar->anterior;
            else
                cola = a_borrar->anterior; // Era la cola
            delete a_borrar; // Liberar memoria
            longitud--;
            eliminado = true;
        } else {
            actual = actual->siguiente;
        }
    }
    return eliminado;
}
```

Libreto:

"Este es el método de eliminación más poderoso: `remove_if`. Recibe un predicado — una función lambda — que define qué condición debe cumplir el nodo para ser eliminado. Recorre la lista nodo por nodo. Cuando encuentra uno que cumple la condición, reconecta los nodos vecinos entre sí: el anterior apunta al siguiente del eliminado, y el siguiente apunta al anterior del eliminado. Luego libera la memoria con `delete`. Esto es lo que hace el sistema cuando, por ejemplo, el usuario elimina una facultad: `datos.facultades.remove_if([id](auto& f){ return f.idFacultad == id; }).`"

CÓDIGO 5 — Destructor: gestión de memoria (Líneas 154–156 + clear)

```
// Líneas 154-156 y 228-237
~ListaEnlazada() {
    clear(); // Llamado automático al salir del scope
}

void clear() noexcept {
    Nodo<T>* actual = cabeza;
    while (actual != nullptr) {
        Nodo<T>* siguiente = actual->siguiente;
        delete actual; // Liberar nodo
        actual = siguiente; // Avanzar al siguiente antes de que desaparezca
    }
    cabeza = cola = nullptr;
    longitud = 0;
}
```

Libreto:

"El destructor garantiza que no haya fugas de memoria. Cuando la lista sale del alcance — por ejemplo, al cerrar el programa — C++ llama automáticamente al destructor, que a su vez llama a `cLear()`. Este recorre la lista nodo por nodo y libera cada uno con `delete`. La clave está en guardar el puntero al siguiente ANTES de eliminar el nodo actual — de lo contrario perderíamos la referencia y la lista quedaría truncada. En Python esto no es necesario porque el recolector de basura lo hace automáticamente, que es precisamente la diferencia más fundamental entre C++ y Python."

7.2 — ARCHIVO: `cpp/persistencia/gestor_persistencia.h` (Líneas 16–46)

¿Por qué mostrarlo? Aquí se ve cómo la lista enlazada se usa en la práctica — TODA la base de datos del sistema es un `struct` lleno de `ListaEnlazada<TipoDeDato>`.

CÓDIGO 6 — El `struct DatosSistema`: toda la base de datos como listas (Líneas 16–47)

```
// cpp/persistencia/gestor_persistencia.h – líneas 16-47
struct DatosSistema {
    ListaEnlazada<Facultad>          facultades;
    ListaEnlazada<ProgramaAcademico> programas;
    ListaEnlazada<Persona>          personas;
    ListaEnlazada<Estudiante>       estudiantes;
    ListaEnlazada<Profesor>         profesores;
    ListaEnlazada<Administrativo>    administrativos;
    ListaEnlazada<Contrato>         contratos;
    ListaEnlazada<MatriculaAcademica> matriculas;
    ListaEnlazada<Calificacion>     calificaciones;
    ListaEnlazada<LiquidacionNomina> liquidacionesNomina;
    ListaEnlazada<PeriodoNomina>    periodosNomina;
    // ... y 15 colecciones más
};
```

Libreto:

"Aquí está la prueba concreta de que usamos la lista en todo el sistema: el `struct DatosSistema` es literalmente la base de datos completa del software. Cada colección — facultades, profesores, estudiantes, contratos, liquidaciones — es una `ListaEnlazada` de su tipo correspondiente. No hay ningún arreglo, ningún `vector` de la librería estándar, ninguna base de datos externa. Todo está en memoria, gestionado por nuestro TAD manual, y persistido en archivos JSON al guardar."

7.3 — ARCHIVO: `cpp/gui/gui_vista_personas.cpp` (Líneas 1338–1416)

¿Por qué mostrarlo? Aquí se ve el CRUD real en acción: cómo se crea una persona y se inserta en la lista.

CÓDIGO 7 — Inserción real de una Persona a la lista (Líneas 1327–1357)

```
// cpp/gui/gui_vista_personas.cpp – líneas 1327-1357
// Cuando el usuario presiona "Guardar" en el modal de Nueva Persona:
Persona p;
p.idPersona    = maxId + 1;          // ID único autoincrementado
p.tipoDocumento = pTipoDoc;          // CC, TI, CE...
p.primerNombre = pPrimerNombre;
p.primerApellido = pPrimerApellido;
p.estado = "ACTIVO";

ctrl.datos.personas.push_back(p);    // ← INSERCIÓN EN LA LISTA ENLAZADA
```

```
// Si el rol es Estudiante, también se inserta en la lista de estudiantes:
if (pRolSeleccionado == 1) {
    Estudiante est;
    est.idEstudiante = maxEstId + 1;
    est.idPersona = idPersFinal; // Llave foránea a la persona
    est.codigoEstudiante = estCodigo;
    ctrl.datos.estudiantes.push_back(est); // ← SEGUNDA INSERCIÓN EN LISTA
}
```

Libreto:

"Este es el CRUD en acción. Cuando el usuario completa el formulario de nueva persona y presiona Guardar, el sistema construye un objeto *Persona* con los datos del formulario, le asigna un ID único autoincrementado recorriendo la lista para encontrar el máximo, y llama a `ctrl.datos.personas.push_back(p)` — inserción directa en la lista enlazada de personas. Si el rol es estudiante, inmediatamente también llama a `ctrl.datos.estudiantes.push_back(est)` con una llave foránea *idPersona* que vincula al estudiante con su persona. Esto implementa exactamente los requerimientos de creación, inclusión y unicidad que exige el taller."

7.4 — COMPARACIÓN: Python NO usa lista enlazada propia (mostrar la diferencia)

¿Por qué mostrarlo? Porque demuestra que son dos implementaciones REALMENTE distintas.

En Python (*dominio/modelo_datos.py*) — Usan clases de datos normales:

```
# En Python, las colecciones son listas nativas de Python []
# gui_controller.py – el controlador guarda todo en listas nativas
self.personas: list[Persona] = []      # Lista nativa Python – O(1) append
self.profesores: list[Profesor] = []   # No hay nodos ni punteros
self.liquidaciones: list[LiquidacionNomina] = []
```

En C++ (*gestor_persistencia.h*) — Usan el TAD propio:

```
// En C++, las mismas colecciones son ListaEnlazada<T> manual
ListaEnlazada<Persona>      personas;    // Nodos con punteros propios
ListaEnlazada<Profesor>     profesores;  // Gestión de memoria manual
ListaEnlazada<LiquidacionNomina> liquidacionesNomina;
```

Libreto para la comparación en vivo (Segmento 8:45–9:30):

"[Mostrar Python en VS Code o en pantalla] Esta es la diferencia estructural entre las dos implementaciones. En Python, el controlador usa listas nativas de Python — que internamente son arreglos dinámicos resizable. El lenguaje gestiona la memoria automáticamente.

[Alt+Tab a VS Code con el C++] En C++, exactamente las mismas colecciones son *ListaEnlazada<T>* — nuestro TAD manual. Cada elemento es un *Nodo<T>* con punteros *anterior* y *siguiente* que conectamos y desconectamos manualmente. Nosotros somos responsables de llamar *delete* en cada nodo cuando se elimina un elemento para evitar fugas de memoria.

La lógica de negocio — el cálculo del salario, la detección EBRA, las restricciones de contratos — es idéntica en ambas versiones. Lo que cambia es la capa de datos: lista nativa en Python, lista enlazada manual con punteros en C++. Eso es exactamente lo que evaluamos en esta materia."

 7.5 — RESUMEN: ARCHIVOS DE CÓDIGO A TENER ABIERTOS EN VS CODE DURANTE EL VIDEO

Pestaña VS Code	Archivo	Líneas Clave	Cuándo Mostrarlo
Tab 1	<code>cpp/dominio/lista_enlazada.h</code>	L11-19 (Nodo), L22-26 (clase), L166-176 (push_back), L311-340 (remove_if), L228-237 (clear)	Segmento 6:00–7:15
Tab 2	<code>cpp/persistencia/gestor_persistencia.h</code>	L16-47 (struct DatosSistema con todas las listas)	Segmento 6:45–7:00 (dentro del bloque de listas)
Tab 3	<code>cpp/gui/gui_vista_personas.cpp</code>	L1327-1357 (push_back de persona y estudiante en el CRUD)	Segmento 7:00–7:15 (cierre del bloque de listas)
Tab 4	<code>ui_gui/gui_controller.py</code>	Líneas con <code>self.personas = []</code> , <code>self.profesores = []</code>	Segmento 8:45–9:00 (comparación Python vs C++)

Consejo práctico: En VS Code, usar `Ctrl + G` para saltar directamente a un número de línea. Por ejemplo: `Ctrl+G` → escribir `166` → Enter, para ir directamente al `push_back`.